ADVANCED WEB PROGRAMMING
AUTHENTICATION & AUTHORIZATION

KHEN MUHAMMAD CAHYO

244107023002

TI-2I

STATE POLYTECHNIC OF MALANG

INFORMATION TECHNOLOGY DEPARTMENT

ASSIGNMENT 1 – AUTHENTICATION IMPLEMENTATION

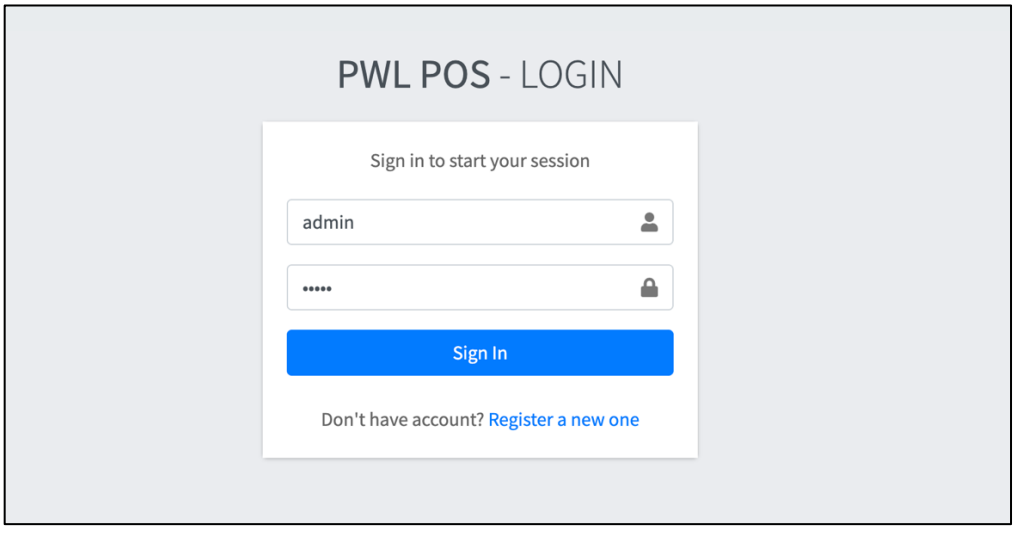
Codes:

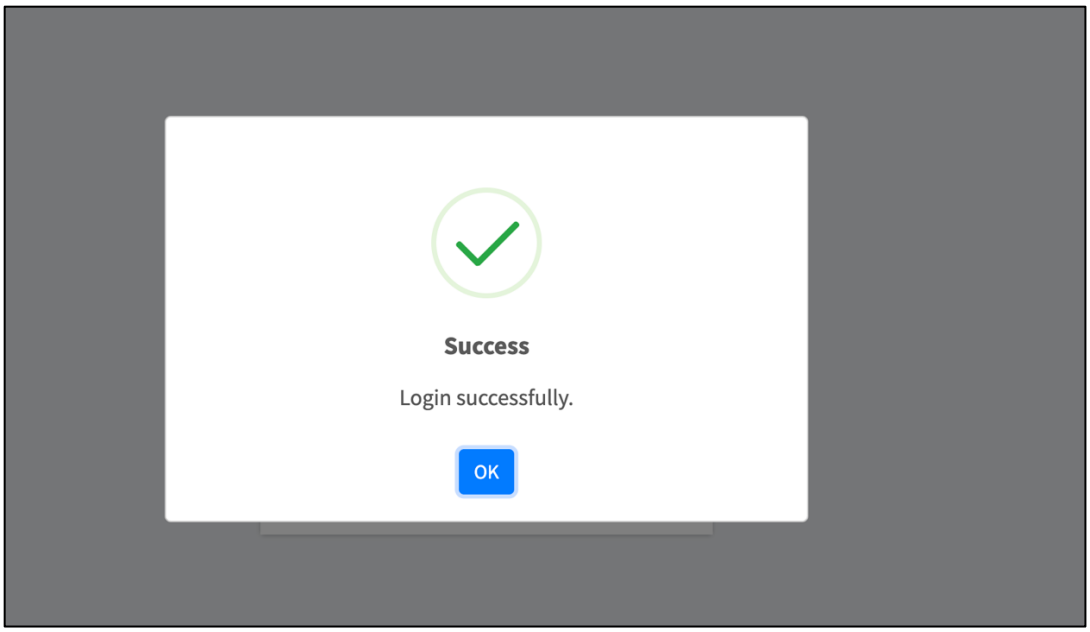
```php
class LoginController extends Controller
{
    Codeium: Refactor | Explain | Generate Function Comment | ×
    public function index () {
        if (Auth::check()) {
            return redirect()->route('dashboard');
        }

        return view('pages.auth.login');
    }

    Codeium: Refactor | Explain | Generate Function Comment | ×
    public function action(LoginRequest $request) {
        if (!$request->validated()) {
            return response()->json([
                'success' => false,
                'message' => 'Failed to login. Please check again your data.',
                'errors' => $request->errors(),
            ]);
        }

        $user = User::where('username', $request->username)->first();

        if (!$user) {
            return response()->json([
                'success' => false,
                'message' => 'Username or password are incorrect.',
            ]);
        }

        if (!password_verify($request->password, $user->password)) {
            return response()->json([
                'success' => false,
                'message' => 'Username or password are incorrect.',
            ]);
        }

        Auth::login($user);

        return response()->json([
            'success' => true,
            'message' => 'Login successfully.',
            'data' => $user
        ]);
    }
}
```

```php
class LoginRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to
     */
    Codeium: Refactor | Explain | ×
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to
     *
     * @return array<string, \Illuminate\Contr
     */
    Codeium: Refactor | Explain | ×
    public function rules(): array
    {
        return [
            'username' => 'required',
            'password' => 'required'
        ];
    }
}
```

```php
Route::prefix('/auth')->group(function () {
    Route::prefix('/login')->group(function () {
        Route::get('', [LoginController::class, 'index'])->name('auth.login-page');
        Route::post('', [LoginController::class, 'action'])->name('auth.login-action');
    });
```

Results:





Explanation:

This code implements basic authentication in Laravel using a login form with validation on both the client and server side. LoginController handles the authentication process, where the index method checks if the user is logged in and redirects him to the dashboard if he is, while the action method validates the input, checks the user's credentials in the database, and authenticates with Auth::login. LoginRequest ensures that username and password inputs are required. Routes defines the route for the login page and authentication process. Blade template (login.blade.php) displays the login form with validation and error notification using session. On the frontend, there is a jQuery script that validates the form before submitting with AJAX, handles validation errors, and displays notifications using SweetAlert. If the login is successful, the user is redirected to the main page.

ASSIGNMENT 2 – AUTHORIZATION IMPLEMENTATION

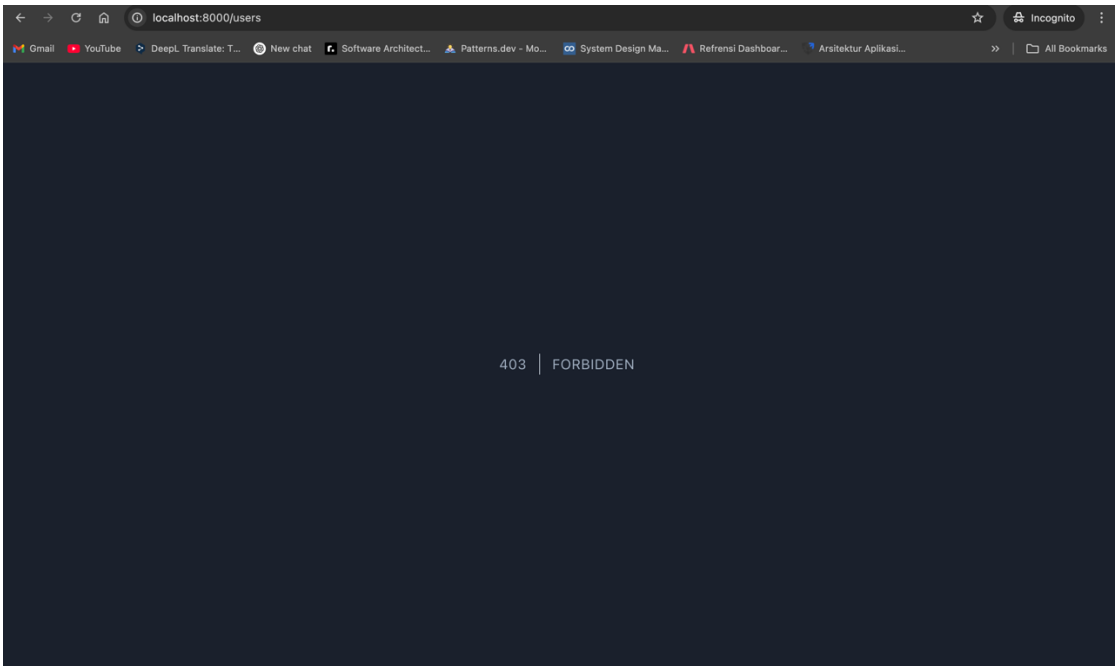
Codes:

```php
Codeium: Refactor | Explain | Generate Function Comment | ×
public function level(): BelongsTo {
    return $this->belongsTo(Level::class, 'level_id', 'level_id');
}

Codeium: Refactor | Explain | Generate Function Comment | ×
public function hasLevel(string $level): bool {
    return $this->level->level_code === $level;
}

Codeium: Refactor | Explain | Generate Function Comment | ×
public function getLevel(): string {
    return $this->level->level_code;
}
```

```php
class AuthorizeLevel
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\
     */

    Codeium: Refactor | Explain | Generate Function Comment | ×
    public function handle(Request $request, Closure $next, $level = ''): Response
    {
        if ($request->user()->hasLevel($level)) {
            return $next($request);
        }

        return abort(403);
    }
}
```

```php
// Route with Admin level middleware
Route::middleware(['authorize-level:ADM'])->group(function () {
    // User Routes
    Route::prefix('/users')->group(function () {
        Route::get('', [UserController::class, 'page'])->name('users.page');
        Route::get('/list', [UserController::class, 'list'])->name('users.list');
        Route::get('/store', [UserController::class, 'storePage'])->name('users.store-page');
        Route::get('/{id}', [UserController::class, 'show'])->name('users.show');
        Route::get('/{id}/show-ajax', [UserController::class, 'showAjax'])->name('users.show-ajax');
        Route::get('/{id}/update', [UserController::class, 'updatePage'])->name('users.update-page');
        Route::patch('/{id}/update', [UserController::class, 'update'])->name('users.update');
        Route::patch('/{id}/update-ajax', [UserController::class, 'updateAjax'])->name('users.update-ajax');
        Route::delete('/{id}/delete', [UserController::class, 'delete'])->name('users.delete');
        Route::delete('/{id}/delete-ajax', [UserController::class, 'deleteAjax'])->name('users.delete-ajax');
        Route::post('/store', [UserController::class, 'store'])->name('users.store');
        Route::post('/store-ajax', [UserController::class, 'storeAjax'])->name('users.store-ajax');
    });
    // Level Routes
    Route::prefix('/levels')->group(function () {
        Route::get('', [LevelController::class, 'page'])->name('levels.page');
        Route::get('/list', [LevelController::class, 'list'])->name('levels.list');
        Route::get('/store', [LevelController::class, 'storePage'])->name('levels.store-page');
        Route::get('/{id}', [LevelController::class, 'show'])->name('levels.show');
        Route::get('/{id}/show-ajax', [LevelController::class, 'showAjax'])->name('levels.show-ajax');
        Route::get('/{id}/update', [LevelController::class, 'updatePage'])->name('levels.update-page');
        Route::patch('/{id}/update', [LevelController::class, 'update'])->name('levels.update');
        Route::patch('/{id}/update-ajax', [LevelController::class, 'updateAjax'])->name('levels.update-ajax');
        Route::delete('/{id}/delete', [LevelController::class, 'delete'])->name('levels.delete');
        Route::delete('/{id}/delete-ajax', [LevelController::class, 'deleteAjax'])->name('levels.delete-ajax');
        Route::post('/store', [LevelController::class, 'store'])->name('levels.store');
        Route::post('/store-ajax', [LevelController::class, 'storeAjax'])->name('levels.store-ajax');
    });
});
```

Results:



Explanation:

This code implements basic authorization in Laravel using the AuthorizeLevel middleware to restrict access based on the user's level. The User model has a relationship with the Level table, where the hasLevel method checks if the user has a certain level, and getLevel returns the user's level code. The AuthorizeLevel middleware checks the user's level before allowing access to the protected route; if the level does not match, the request is blocked with HTTP 403 (Forbidden). Routing uses the authorize-level:ADM middleware to protect the /users group of routes, ensuring only users with ADM level can access them. These routes include CRUD operations for users, both in the form of pages and AJAX.

ASSIGNMENT 3 – MULTI-LEVEL AUTHORIZATION IMPLEMENTATION

Codes:

Additional Codes:

- Middleware for check user authentication.
- Authorization on sidebar menu based on user level.

```php
class AuthorizeLevel
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)  $next
     */
    Codeium: Refactor | Explain | ×
    public function handle(Request $request, Closure $next, ...$levels): Response
    {
        $userLevel = $request->user()->getLevel();

        if (in_array($userLevel, $levels)) {
            return $next($request);
        }

        return abort(403);
    }
}
```

```php
class CheckAuth
{
    /**
     * Handle an incoming request.
     *
     * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)  $
     */
    Codeium: Refactor | Explain | ×
    public function handle(Request $request, Closure $next): Response
    {
        if (!Auth::check()) {
            return redirect()->route('auth.login-page')->with('error', 'You must login first.');
        }

        return $next($request);
    }
}
```

```php
Route::middleware(['check-auth'])->group(function () {
    Route::get('/', [DashboardController::class, 'index'])->name('dashboard');
    // Route with Admin level middleware
    Route::middleware(['authorize-level:ADM'])->group(function () {
        // User Routes
        Route::prefix('/users')->group(function () {
            Route::get('', [UserController::class, 'page'])->name('users.page');
            Route::get('/list', [UserController::class, 'list'])->name('users.list');
            Route::get('/store', [UserController::class, 'storePage'])->name('users.store-page');
            Route::get('/{id}', [UserController::class, 'show'])->name('users.show');
            Route::get('/{id}/show-ajax', [UserController::class, 'showAjax'])->name('users.show-ajax');
            Route::get('/{id}/update', [UserController::class, 'updatePage'])->name('users.update-page');
            Route::patch('/{id}/update', [UserController::class, 'update'])->name('users.update');
            Route::patch('/{id}/update-ajax', [UserController::class, 'updateAjax'])->name('users.update-ajax');
            Route::delete('/{id}/delete', [UserController::class, 'delete'])->name('users.delete');
            Route::delete('/{id}/delete-ajax', [UserController::class, 'deleteAjax'])->name('users.delete-ajax');
            Route::post('/store', [UserController::class, 'store'])->name('users.store');
            Route::post('/store-ajax', [UserController::class, 'storeAjax'])->name('users.store-ajax');
        });
        // Level Routes
        Route::prefix('/levels')->group(function () {
            Route::get('', [LevelController::class, 'page'])->name('levels.page');
            Route::get('/list', [LevelController::class, 'list'])->name('levels.list');
            Route::get('/store', [LevelController::class, 'storePage'])->name('levels.store-page');
            Route::get('/{id}', [LevelController::class, 'show'])->name('levels.show');
            Route::get('/{id}/show-ajax', [LevelController::class, 'showAjax'])->name('levels.show-ajax');
            Route::get('/{id}/update', [LevelController::class, 'updatePage'])->name('levels.update-page');
            Route::patch('/{id}/update', [LevelController::class, 'update'])->name('levels.update');
            Route::patch('/{id}/update-ajax', [LevelController::class, 'updateAjax'])->name('levels.update-ajax');
            Route::delete('/{id}/delete', [LevelController::class, 'delete'])->name('levels.delete');
            Route::delete('/{id}/delete-ajax', [LevelController::class, 'deleteAjax'])->name('levels.delete-ajax');
            Route::post('/store', [LevelController::class, 'store'])->name('levels.store');
            Route::post('/store-ajax', [LevelController::class, 'storeAjax'])->name('levels.store-ajax');
        });
    });
```

```php
// Route with Admin or Staff level middleware
Route::middleware(['authorize-level:ADM,MNG'])->group(function () {
    // Category Routes
    Route::prefix('/categories')->group(function () {
        Route::get('', [CategoryController::class, 'page'])->name('categories.page');
        Route::get('/list', [CategoryController::class, 'list'])->name('categories.list');
        Route::get('/store', [CategoryController::class, 'storePage'])->name('categories.store-page');
        Route::get('/{id}', [CategoryController::class, 'show'])->name('categories.show');
        Route::get('/{id}/show-ajax', [CategoryController::class, 'showAjax'])->name('categories.show-ajax');
        Route::get('/{id}/update', [CategoryController::class, 'updatePage'])->name('categories.update-page');
        Route::patch('/{id}/update', [CategoryController::class, 'update'])->name('categories.update');
        Route::patch('/{id}/update-ajax', [CategoryController::class, 'updateAjax'])->name('categories.update-ajax');
        Route::delete('/{id}/delete', [CategoryController::class, 'delete'])->name('categories.delete');
        Route::delete('/{id}/delete-ajax', [CategoryController::class, 'deleteAjax'])->name('categories.delete-ajax');
        Route::post('/store', [CategoryController::class, 'store'])->name('categories.store');
        Route::post('/store-ajax', [CategoryController::class, 'storeAjax'])->name('categories.store-ajax');
    });
    // Item Routes
    Route::prefix('/items')->group(function () {
        Route::get('', [ItemController::class, 'page'])->name('items.page');
        Route::get('/list', [ItemController::class, 'list'])->name('items.list');
        Route::get('/store', [ItemController::class, 'storePage'])->name('items.store-page');
        Route::get('/{id}', [ItemController::class, 'show'])->name('items.show');
        Route::get('/{id}/show-ajax', [ItemController::class, 'showAjax'])->name('items.show-ajax');
        Route::get('/{id}/update', [ItemController::class, 'updatePage'])->name('items.update-page');
        Route::patch('/{id}/update', [ItemController::class, 'update'])->name('items.update');
        Route::patch('/{id}/update-ajax', [ItemController::class, 'updateAjax'])->name('items.update-ajax');
        Route::delete('/{id}/delete', [ItemController::class, 'delete'])->name('items.delete');
        Route::delete('/{id}/delete-ajax', [ItemController::class, 'deleteAjax'])->name('items.delete-ajax');
        Route::post('/store', [ItemController::class, 'store'])->name('items.store');
        Route::post('/store-ajax', [ItemController::class, 'storeAjax'])->name('items.store-ajax');
    });
    // Stock Routes
    Route::prefix('/stocks')->group(function () {
        Route::get('', [StockController::class, 'page'])->name('stocks.page');
        Route::get('/list', [StockController::class, 'list'])->name('stocks.list');
        Route::get('/store', [StockController::class, 'storePage'])->name('stocks.store-page');
        Route::get('/{id}', [StockController::class, 'show'])->name('stocks.show');
        Route::get('/{id}/show-ajax', [StockController::class, 'showAjax'])->name('stocks.show-ajax');
        Route::get('/{id}/update', [StockController::class, 'updatePage'])->name('stocks.update-page');
        Route::patch('/{id}/update', [StockController::class, 'update'])->name('stocks.update');
        Route::patch('/{id}/update-ajax', [StockController::class, 'updateAjax'])->name('stocks.update-ajax');
        Route::delete('/{id}/delete', [StockController::class, 'delete'])->name('stocks.delete');
        Route::delete('/{id}/delete-ajax', [StockController::class, 'deleteAjax'])->name('stocks.delete-ajax');
        Route::post('/store', [StockController::class, 'store'])->name('stocks.store');
        Route::post('/store-ajax', [StockController::class, 'storeAjax'])->name('stocks.store-ajax');
    });
});
```

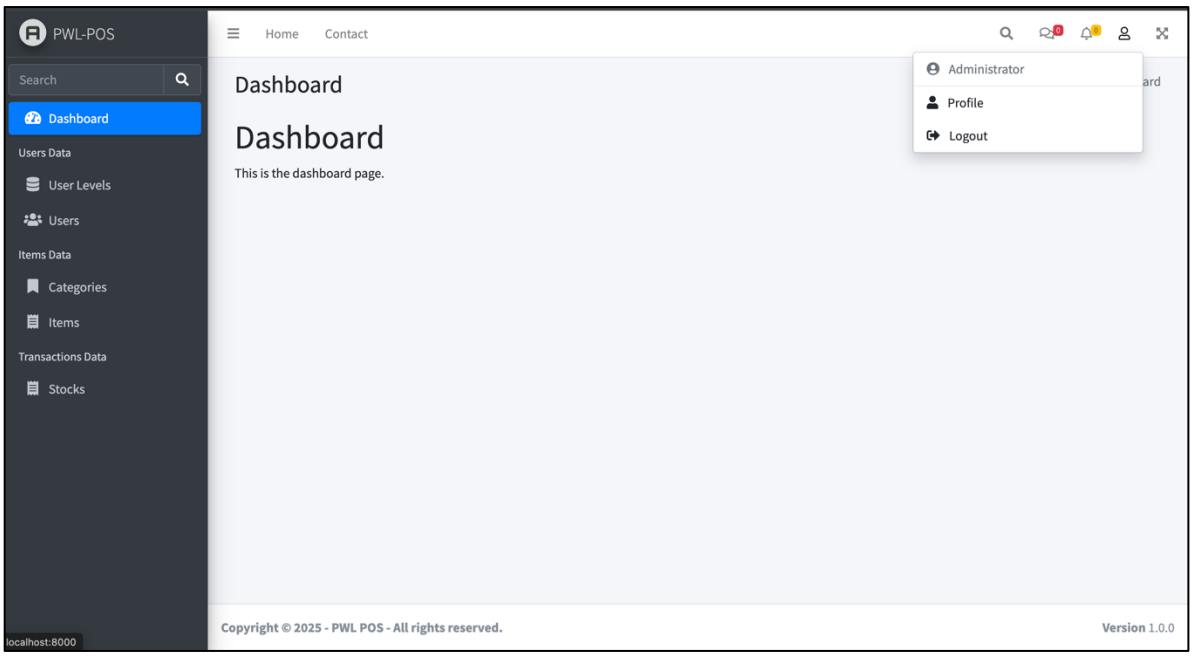
```php
@php
    $sidebarMenus = [
        [
            'group' => null,
            'menus' => [
                [
                    'name' => 'Dashboard',
                    'route' => 'dashboard',
                    'icon' => 'fas fa-tachometer-alt',
                    'levels' => ['ADM', 'MNG', 'STF'],
                ],
            ],
        ],
        [
            'group' => 'Users Data',
            'menus' => [
                [
                    'name' => 'User Levels',
                    'route' => 'levels.page',
                    'icon' => 'fas fa-database',
                    'levels' => ['ADM'],
                ],
                [
                    'name' => 'Users',
                    'route' => 'users.page',
                    'icon' => 'fas fa-users',
                    'levels' => ['ADM'],
                ],
            ],
        ],
```

```php
        [
            'group' => 'Items Data',
            'menus' => [
                [
                    'name' => 'Categories',
                    'route' => 'categories.page',
                    'icon' => 'fas fa-bookmark',
                    'levels' => ['ADM', 'MNG'],
                ],
                [
                    'name' => 'Items',
                    'route' => 'items.page',
                    'icon' => 'fas fa-receipt',
                    'levels' => ['ADM', 'MNG'],
                ],
            ],
        ],
        [
            'group' => 'Transactions Data',
            'menus' => [
                [
                    'name' => 'Stocks',
                    'route' => 'stocks.page',
                    'icon' => 'fas fa-receipt',
                    'levels' => ['ADM', 'MNG'],
                ],
                [
                    'name' => 'Transactions',
                    'route' => null,
                    'icon' => 'fas fa-money-bill',
                    'levels' => ['STF'],
                ],
            ],
        ],
    ];
@endphp
```

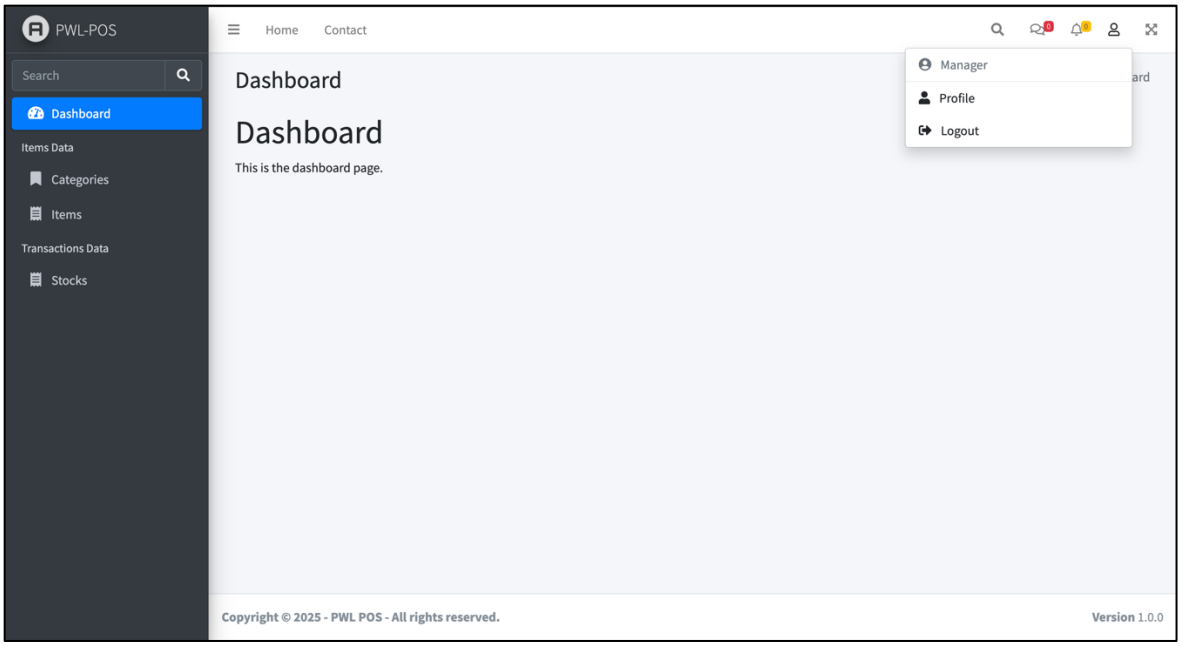
```php
<!-- Sidebar Menu -->
<nav class="mt-2">
    <ul class="nav nav-pills nav-sidebar flex-column" data-widget="treeview" role="menu"
        data-accordion="false">
        @foreach ($sidebarMenus as $sidebarMenu)
            @php
                $visibleMenus = array_filter($sidebarMenu['menus'], function ($menu) {
                    return isset($menu['levels']) && in_array(Auth::user()->level->level_code, $menu['levels']);
                });
            @endphp

            @if (count($visibleMenus) > 0)
                @if ($sidebarMenu['group'])
                    <li class="nav-header">{{ $sidebarMenu['group'] }}</li>
                @endif

                @foreach ($visibleMenus as $menu)
                    <li class="nav-item">
                        <a href="{{ $menu['route'] !== null ? route($menu['route']) : '#' }}"
                            @class(['nav-link', 'active' => request()->routeIs($menu['route'])])>
                            <i class="nav-icon {{ $menu['icon'] }}"></i>
                            <p>{{ $menu['name'] }}</p>
                        </a>
                    </li>
                @endforeach
            @endif
        @endforeach
    </ul>
</nav>
<!-- End of Sidebar Menu -->
```

Results:

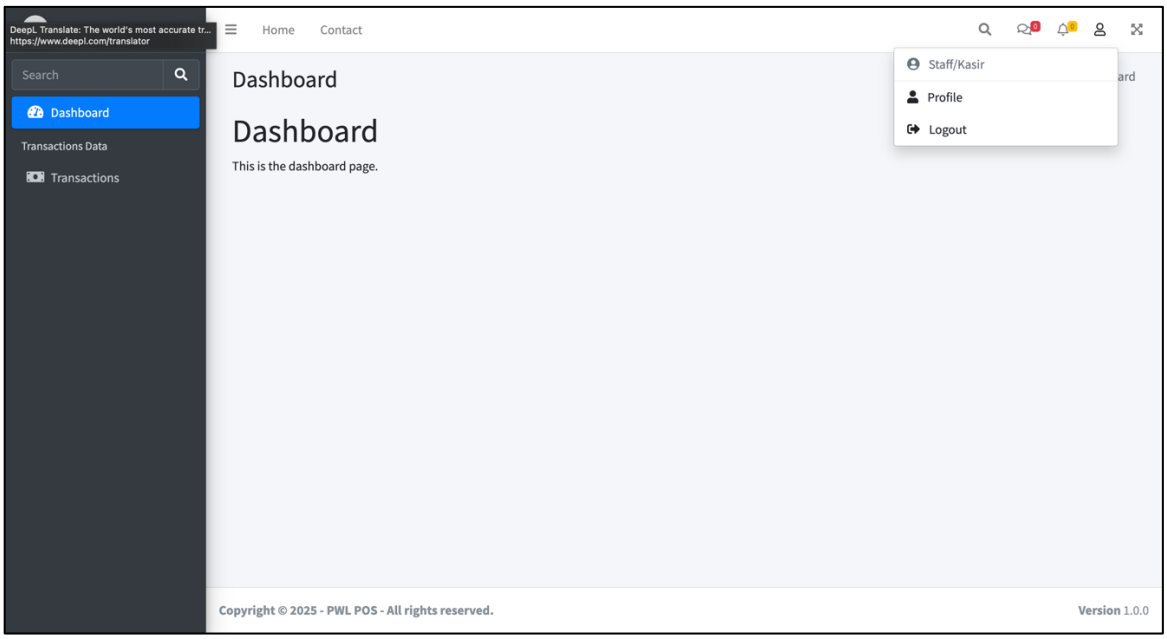
When level Admin:

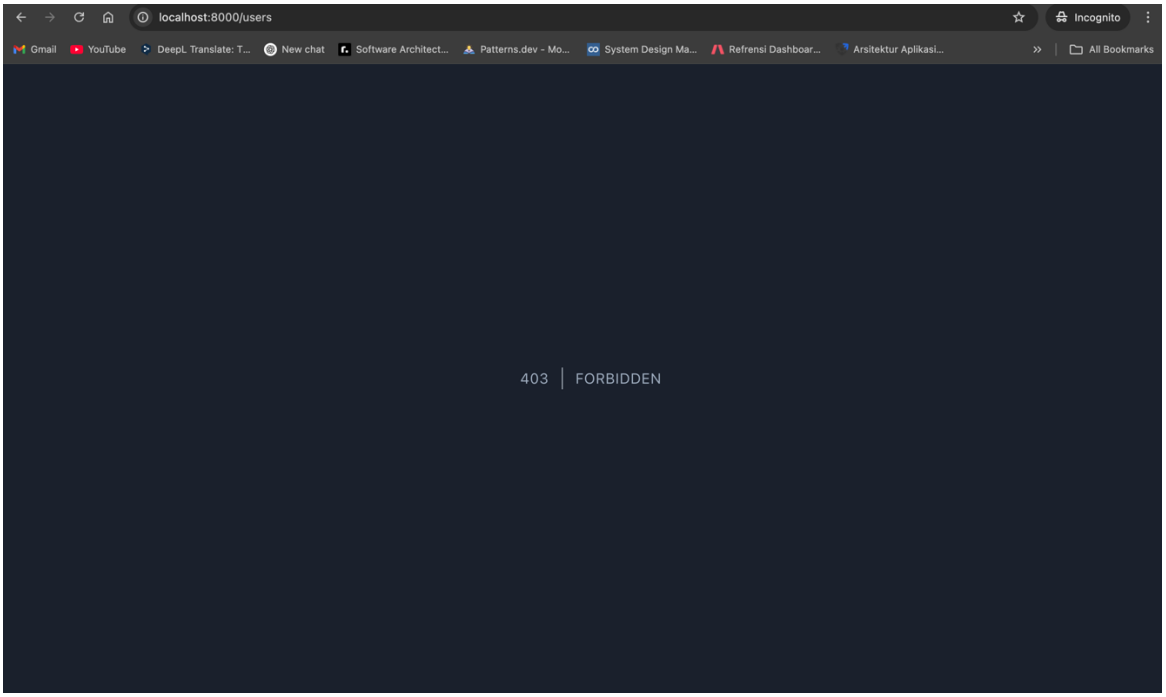


When level Manager:



When level Staff:





Explanation:

The code provided consists of authorization middleware, menu sidebar structure, and route registration in Laravel. The AuthorizeLevel middleware checks whether the user has the appropriate access level before accessing a route by comparing the user level from request()->user()->getLevel() against the list of allowed levels. If the user's level is not included in the list, the request will be blocked with a 403 Forbidden code. In addition, there is a CheckAuth middleware that ensures that users are logged in before accessing certain pages, if not, they will be directed to the login page with the error message “You must login first.”. In the sidebar menu section, there is a $sidebarMenus array that contains a list of menus that can be accessed by users based on their level, where each menu has a label, icon, and a list of submenus if any. Then, in route registration, Laravel uses middleware groups to restrict user access based on predefined authorizations. These routes use CheckAuth middleware to ensure only logged-in users can access them as well as AuthorizeLevel middleware to restrict access based on user level. With this approach, Laravel can flexibly and securely manage user access within the application.

ASSIGNMENT 4 – REGISTER FORM

Codes:

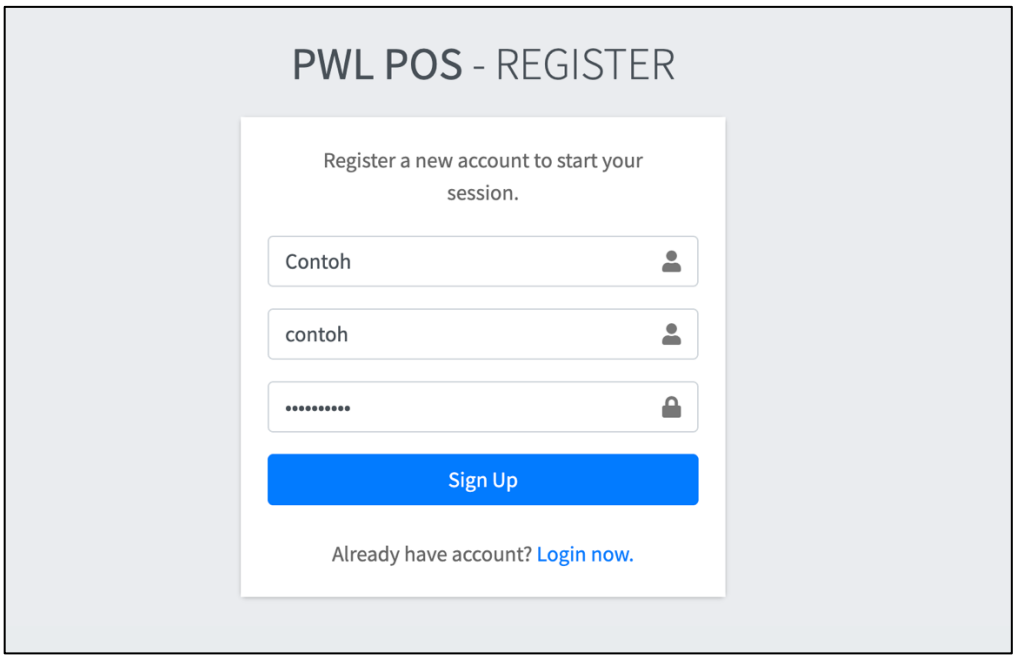
```php
class RegisterController extends Controller
{
    Codeium: Refactor | Explain | Generate Function Comment | ×
    public function index () {
        if (Auth::check()) {
            return redirect()->route('dashboard');
        }

        return view('pages.auth.register');
    }

    Codeium: Refactor | Explain | Generate Function Comment | ×
    public function action (RegisterRequest $request) {
        if (!$request->validated()) {
            return response()->json([
                'success' => false,
                'message' => 'Failed to register. Please check again your data.',
                'errors' => $request->errors(),
            ]);
        }

        $defaultLevel = Level::where('level_code', 'STF')->first();
        User::create([
            'level_id' => $defaultLevel->level_id,
            'name' => $request->name,
            'username' => $request->username,
            'password' => bcrypt($request->password),
        ]);

        return response()->json([
            'success' => true,
            'message' => 'Register successfully.',
        ]);
    }
}
```

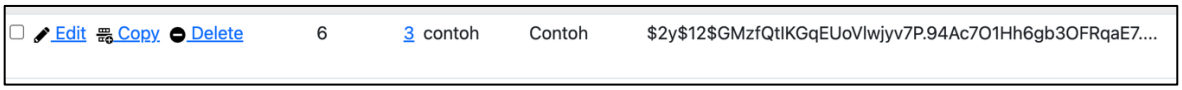
```php
Codeium: Refactor | Explain
class RegisterRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    Codeium: Refactor | Explain | ×
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, \Illuminate\Contracts\Validation\Validation
     */
    Codeium: Refactor | Explain | ×
    public function rules(): array
    {
        return [
            'name' => 'required|max:100',
            'username' => 'required|max:20|unique:users,username',
            'password' => 'required|min:8',
        ];
    }
}
```

```blade
@push('scripts')
    <script>
        $(document).ready(function() {
            $('#registerForm').validate({
                rules: {
                    name: {
                        required: true,
                        maxlength: 100
                    },
                    username: {
                        required: true,
                        maxlength: 20
                    },
                    password: {
                        required: true,
                        minlength: 8
                    },
                },
                submitHandler: function (form) {
                    $.ajax({
                        url: form.action,
                        method: form.method,
                        data: $(form).serialize(),
                        success: function (response) {
                            if (response.success) {
                                Swal.fire({
                                    icon: 'success',
                                    title: 'Success',
                                    text: response.message,
                                }).then(() => {
                                    window.location.href = '/auth/login';
                                });
                            } else {
                                $('.error-text').text('');
                                $.each(response.errors, function(key, value) {
                                    $('#error-' + key).text(value);
                                });

                                Swal.fire({
                                    icon: 'error',
                                    title: 'Error',
                                    text: response.message,
                                });
                            }
                        },
                        error: function (xhr) {
                            Swal.fire({
                                icon: 'error',
                                title: 'Error',
                                text: xhr.responseJSON.message,
                            });
                        }
                    });

                    return false;
                },
                errorElement: 'small',
                errorPlacement: function(error, element) {
                    error.addClass('invalid-feedback');
                    element.closest('.input-group').append(error);
                },
                highlight: function(element, errorClass, validClass) {
                    $(element).addClass('is-invalid');
                },
                unhighlight: function(element, errorClass, validClass) {
                    $(element).removeClass('is-invalid');
                }
            });
        });
    </script>
```

```php
Route::prefix('/register')->group(function () {
    Route::get('', [RegisterController::class, 'index'])->name('auth.register-page');
    Route::post('', [RegisterController::class, 'action'])->name('auth.register-action');
});
```

Results:


PWL POS - REGISTER
Register a new account to start your session.
Contoh
contoh
Sign Up
Already have account? Login now.


Edit
Copy
Delete
6
3
contoh
Contoh
$2y$12$GMzfQtIKGqEUoVlwjyv7P.94Ac7O1Hh6gb3OFRqaE7....

Explanation:

This code implements the user registration feature in Laravel, consisting of a controller, form request, and view with validation and AJAX for the registration process. RegisterController has two main methods: index() which displays the registration page if the user is not logged in, and action() which handles the registration process. If the validation of the RegisterRequest fails, a JSON response with an error message is returned. If successful, a user account is created with a default level retrieved from the database, then a success response is sent in JSON format. RegisterRequest serves as input validation with rules such as name must be filled with a maximum of 100 characters, a unique username with a maximum of 20 characters, and a password of at least 8 characters. The registration form view is created using Blade template, with inputs for name, username, and password. If there is an error from the session, an error message will be displayed in the alert. This form uses AJAX to send data to the server without reloading the page. If the registration is successful, SweetAlert will display a success notification and redirect the user to the login page. If it fails, a validation error will be displayed below the corresponding input. In addition, additional validation with jQuery Validate ensures that the input is not empty and meets predefined rules before being sent to the server. With the combination of backend validation in Laravel and frontend validation using jQuery, this system ensures that the submitted data is up to standard before further processing.

GitHub Push Results: